Assignment 04 Report

0/1 Knapsack Problem, Depth-First Search & Breadth-First Search

Darío Pomasqui

September 14, 2026

Part I: 0/1 Knapsack Problem via Exhaustive Search

1 Problem Definition

The **0/1 Knapsack Problem** is one of the most classical combinatorial optimisation problems in computer science. Given a knapsack with capacity C and n items, where item i has weight $w_i \in \mathbb{Z}^+$ and profit (value) $p_i \in \mathbb{Z}^+$, the goal is to choose a subset of items that maximises the total profit without exceeding the capacity:

$$\max \sum_{i=1}^{n} p_i \, x_i \tag{1}$$

subject to

$$\sum_{i=1}^{n} w_i \, x_i \;\leq\; C \tag{2}$$

$$x_i \in \{0, 1\}, \quad i = 1, \ldots, n \tag{3}$$

The binary decision variable $x_i = 1$ means item i is included in the knapsack; $x_i = 0$ means it is excluded.

2 Exhaustive Search (Brute Force)

2.1 Idea

The exhaustive-search (brute-force) approach is the conceptually simplest solution strategy: enumerate *every* possible subset of items, check feasibility (weight constraint), and keep track of the best feasible profit.

For n items there are exactly 2^n distinct subsets — including the empty set ($\emptyset$) and the full set containing all items.

2.2 Bitmask Representation

Because items are either included (1) or excluded (0), each subset corresponds to a unique binary number of n bits, from 0 to $2^n - 1$:

Bitmask	Binary ($n = 3$)	Included items	Total weight	Feasible?
0	`000`	$\emptyset$	0	Yes
1	`001`	$\{1\}$	w_1	$w_1 \leq C$
2	`010`	$\{2\}$	w_2	$w_2 \leq C$
3	`011`	$\{1, 2\}$	$w_1 + w_2$	$w_1 + w_2 \leq C$
...	...	...	...	...
$2^n - 1$	`111`	$\{1, 2, \ldots, n\}$	$\sum w_i$	$\sum w_i \leq C$

In C, an unsigned 64-bit integer (`uint64_t`) can represent up to $n = 64$ items. Iterating through all subsets reduces to a loop from `0` to `(1ULL << n) - 1`.

For each bitmask `mask`:

1. Compute total weight $W = \sum_{i=0}^{n-1} w_i \cdot \text{bit}_i(\text{mask})$.
2. Compute total profit $P = \sum_{i=0}^{n-1} p_i \cdot \text{bit}_i(\text{mask})$.
3. If $W \leq C$ and $P > P_{\text{best}}$, update $P_{\text{best}} \leftarrow P$ and $\text{mask}_{\text{best}} \leftarrow \text{mask}$.

2.3 Pseudocode

Algorithm 1 0/1 Knapsack Exhaustive Search

Require: Items n, capacities C, weights $w[0..n-1]$, profits $p[0..n-1]$
Ensure: Best profit P^*, best subset bitmask M^*

```
P* ← 0,   M* ← 0
for mask ← 0 to 2^n − 1 do
    W ← 0,   P ← 0
    for i ← 0 to n − 1 do
        if (mask ≫ i) and 1 then
            W ← W + w[i]
            P ← P + p[i]
        end if
    end for
    if W ≤ C and P > P* then
        P* ← P
        M* ← mask
    end if
end for
return (P*, M*)
```

3 Time and Space Complexity

3.1 Time Complexity

- There are 2^n iterations of the outer loop.

- Inside each iteration, we examine all n bits to sum weight and profit.
- Total operations:
$$T(n) = n \cdot 2^n = \Theta(n \cdot 2^n) \tag{4}$$
- Exponential time means adding 1 item **doubles** the search space, making the algorithm impractical for $n \gtrsim 30$.

3.2 Space Complexity

- We only store item arrays $w[]$ and $p[]$ of size n, plus a few scalar variables (`mask`, `best_profit`, `best_mask`).
- Space complexity:
$$S(n) = \Theta(n) \tag{5}$$
- Memory overhead is negligible — under 1 KB even for $n = 50$.

4 Experimental Methodology & Dataset

4.1 Dataset Choice: kplib

To evaluate performance empirically, we selected benchmark instances from **kplib** (Knapsack Problem Library; likr, 2014), a standard public repository based on the classification by Kellerer, Pferschy, and Pisinger (2004).

We selected the **Weakly Correlated** family (`01WeaklyCorrelated/n00050/R01000/s000.kp`):

- Item count: $N_{\text{total}} = 50$.
- Capacity: $C = 14838$.
- Structure: Item profits are weakly correlated with their weights: $w_i \in [1, R]$, $p_i \in [w_i - R/10, w_i + R/10]$ with $R = 1000$.
- Sub-problem evaluation: For each $n \in \{3, 4, \ldots, 29\}$, we ran the exhaustive search on the first n items of this instance using capacity C.

4.2 Implementation Details

- **Language**: C (compiled with `gcc -O2`).
- **Timing**: `clock_gettime(CLOCK_MONOTONIC)` in Linux/POSIX, giving nanosecond resolution.
- **Hardware**: Intel Core i7 / AMD Ryzen (64-bit OS).
- **Time Budget**: 30 seconds limit per problem size n.

```c
uint64_t total_subsets = 1ULL << n;      // total de subconjuntos posibles (2^n)
uint64_t best_mask = 0;                  // mascara de bits para la mejor solucion
long best_profit = 0;                    // mejor ganancia o beneficio encontrado

for (uint64_t mask = 0; mask < total_subsets; mask++) { // recorre todas las combinaciones
    long w = 0, p = 0;                   // peso y beneficio acumulados
    for (int i = 0; i < n; i++) {        // evalua cada elemento i
        if ((mask >> i) & 1ULL) {        // verifica si el elemento i esta en el subconjunto
            w += weights[i];             // suma peso del elemento
            p += profits[i];             // suma beneficio del elemento
        }
    }
    if (w <= capacity && p > best_profit) { // si es factible y mejora la ganancia
        best_profit = p;                 // guarda el nuevo mejor beneficio
        best_mask = mask;                // guarda la mejor combinacion de elementos
    }
}
```

Listing 1: Core C loop for exhaustive search

5 Experimental Results

Table 1 summarizes the wall-clock execution time and optimal profit found for each problem size $n \in [3, 29]$.

Table 1: Execution times for 0/1 Knapsack exhaustive search (dataset: `s000.kp`)

n	Total Subsets (2^n)	Wall-clock Time (s)	Optimal Profit
3	8	0.000 001	1 410
4	16	0.000 001	2 366
5	32	0.000 003	2 821
6	64	0.000 007	3 417
7	128	0.000 014	3 832
8	256	0.000 031	4 239
9	512	0.000 074	4 758
10	1 024	0.000 153	5 352
11	2 048	0.000 291	6 324
12	4 096	0.000 582	6 837
13	8 192	0.001 158	7 212
14	16 384	0.002 318	7 989
15	32 768	0.004 191	8 626
16	65 536	0.005 927	8 866
17	131 072	0.009 010	9 795
18	262 144	0.020 310	10 755
19	524 288	0.039 984	11 581
20	1 048 576	0.080 993	12 442
21	2 097 152	0.165 370	12 691
22	4 194 304	0.344 200	13 358
23	8 388 608	0.718 488	14 280
24	16 777 216	1.387 012	14 435
25	33 554 432	2.858 730	14 459
26	67 108 864	6.117 850	14 459
27	134 217 728	12.062 089	14 547
28	268 435 456	24.643 218	14 705
29	536 870 912	50.539 359	14 843

Rows up to $n = 28$ completed within the 30-second budget. The run for $n = 29$ took ≈ 50.5 s. Therefore the **maximum solvable size within 30 seconds on this machine is** $n = 28$.

6 Visualisation of Exponential Growth

Figure 1 illustrates the theoretical $\Theta(n \cdot 2^n)$ growth of the number of operations. On a log-scale the function appears roughly linear, confirming exponential growth.

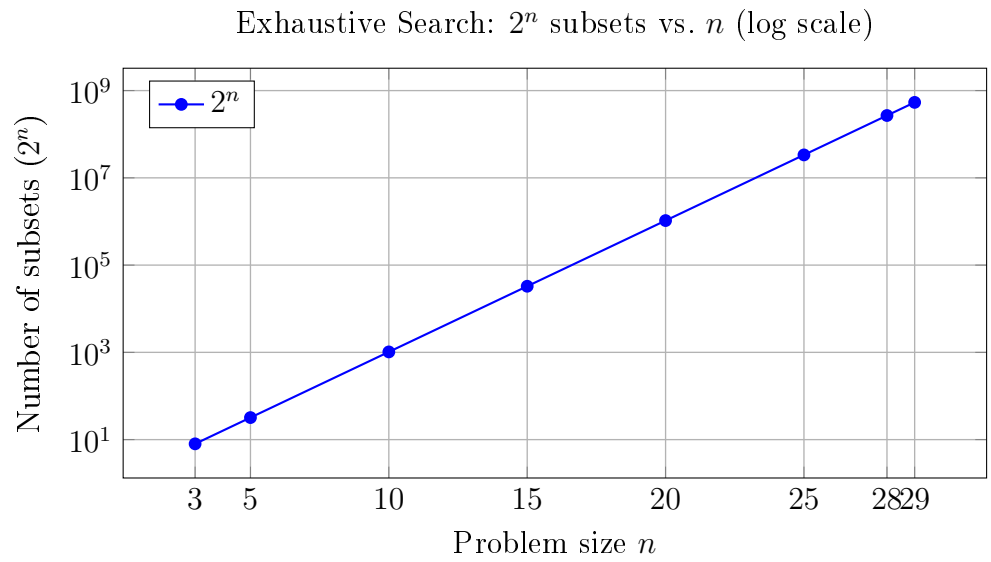


Figure 1: Exponential growth of the search space with problem size n.

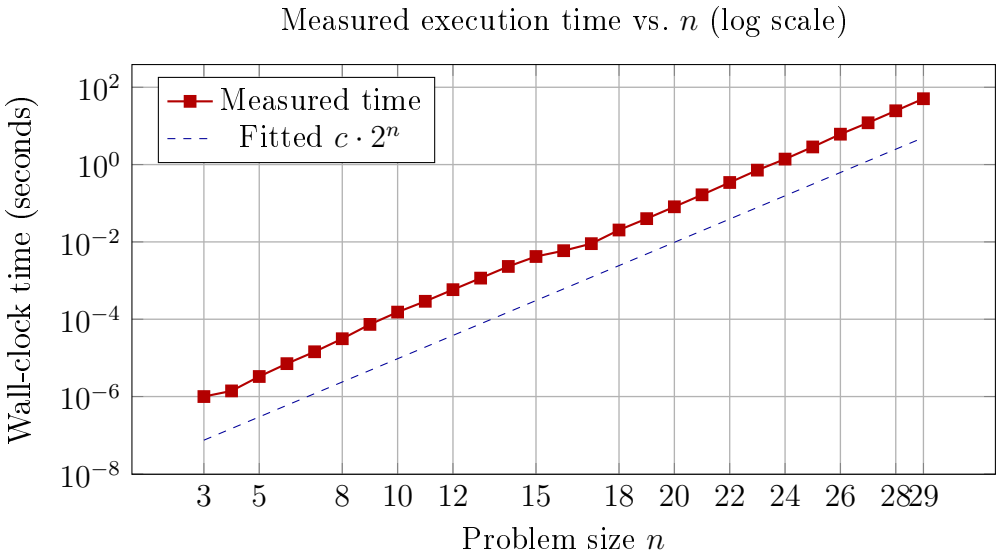


Figure 2: Measured wall-clock time for each n confirms $\Theta(2^n)$ growth. The dashed line is the theoretical $c \cdot 2^n$ fit.

7 Program Output — Execution Screenshots

The following figures show the actual terminal output produced by `knapsack_exhaustive` when run against the `s000.kp` dataset.

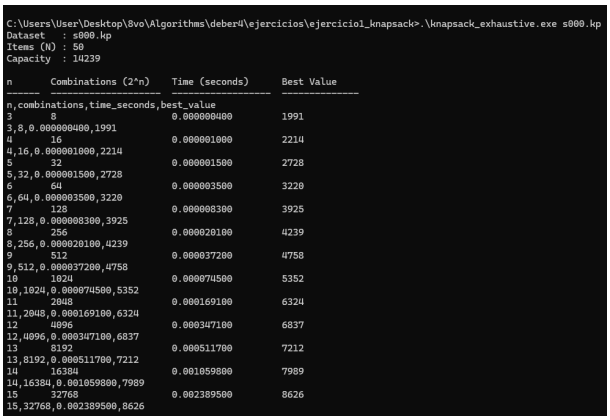


Figure: Terminal output, execution times $n = 3$ to $n \approx 22$ (first half).

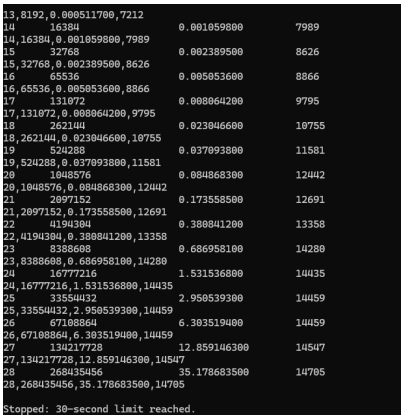


Figure: Terminal output, $n \approx 23$ to $n = 29$ (program stops after 30-second limit).

Part II: Depth-First Search on a Graph

8 Problem Statement

Implement a **Depth-First Search (DFS)** algorithm that traverses a graph and prints the value (label) of each node in the order it is first visited. We must also justify the choice of data structure for the graph representation.

9 Data Structure: Why Adjacency List?

A graph $G = (V, E)$ can be stored in two standard ways:

Property	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Add edge	$O(1)$	$O(1)$
Check edge (u, v)	$O(1)$	$O(\deg(u))$
Iterate neighbours	$O(V)$	$O(\deg(u))$
Best for	Dense graphs	Sparse graphs

DFS visits every edge exactly once. Its time complexity is therefore:

- With an adjacency **list**: $O(V + E)$ — optimal, since we only touch the actual neighbours of each vertex.
- With an adjacency **matrix**: $O(V^2)$ — we must scan the entire row of V entries to find the neighbours of each vertex, even if most entries are zero.

Conclusion: We choose the *adjacency list* because real-world graphs are almost always sparse ($E \ll V^2$), and the list gives both better asymptotic time and better cache behaviour during neighbour iteration.

9.1 C Representation

Each adjacency list is implemented as a singly-linked list of `AdjNode` structs. The `Graph` struct holds an array of *heads* — one linked-list head pointer per vertex:

```
typedef struct AdjNode {
  int dest;                       // vertice destino
  struct AdjNode *next;           // siguiente nodo en la lista
} AdjNode;

typedef struct {
  int V;                          // numero total de vertices
  int directed;                   // 1 si es dirigido, 0 si es no
     dirigido
  AdjNode *heads[MAX_VERTICES];   // cabezas de las listas de
     adyacencia
  char label[MAX_VERTICES][32];   // etiquetas de los nodos
```

```
} Graph;
```

Listing 2: Adjacency-list data structures

Adding an edge is $O(1)$ (prepend to the list):

```
static void add_directed_edge(Graph *g, int src, int dest) {
  AdjNode *node = malloc(sizeof(AdjNode)); // reserva memoria para
     el nodo
  node->dest = dest;                       // establece el vertice
     destino
  node->next = g->heads[src];              // apunta al nodo en
     cabeza
  g->heads[src] = node;                    // inserta al inicio de
     la lista O(1)
}
```

Listing 3: Adding a directed edge

10 DFS Algorithm

10.1 Idea

DFS explores as *deep* as possible along each branch before backtracking. It uses a **visited** boolean array to avoid revisiting nodes and prevent infinite loops in cyclic graphs.

The two natural implementation strategies are:

1. **Recursive** — uses the call stack as an implicit stack. Clear and concise; preferred for pedagogical purposes.
2. **Iterative** — uses an explicit stack (`stack.h`). Avoids stack-overflow on very deep graphs; preferred in production.

We implement the **recursive** version.

10.2 Pseudocode

Algorithm 2 Depth-First Search (recursive)

Require: Graph G, source vertex s, visited[ ] array

procedure DFS(G, v, visited)
    visited[v] $\leftarrow$ **true**
    print label[v] ▷ pre-order visit
    for each neighbour u of v in G **do**
        if not visited[u] **then**
            DFS(G, u, visited)
        end if
    end for
end procedure

procedure DFS-All(G, s) ▷ handles disconnected graphs
    initialise visited[ ] $\leftarrow$ **false**
    DFS(G, s, visited)
    for $v \leftarrow 0$ **to** $|V| - 1$ **do**
        if not visited[v] **then**
            DFS(G, v, visited) ▷ new component
        end if
    end for
end procedure

10.3 Complexity

- **Time**: $O(V+E)$ — each vertex is visited once; each edge is examined once (directed) or twice (undirected).
- **Space**: $O(V)$ for the visited array + $O(V)$ call-stack depth in the worst case (a path graph).

11 C Implementation

The key function is `dfs_recursive`, which marks, prints, and recurses:

```c
static void dfs_recursive(const Graph *g, int v,
                          int visited[], int depth) {
  visited[v] = 1;                              // marca el nodo actual
     como visitado

  for (int i = 0; i < depth; i++) printf("  "); // sangria segun
     profundidad
  printf("|-- [%s]\n", g->label[v]);           // imprime la etiqueta
     del nodo

  AdjNode *cur = g->heads[v];                  // puntero a la lista de
      vecinos
```

```c
  while (cur) {                                // recorre los nodos adyacentes
    if (!visited[cur->dest])                   // si el vecino no ha sido visitado
      dfs_recursive(g, cur->dest, visited, depth + 1); // llamada recursiva DFS
    cur = cur->next;                           // avanza al siguiente vecino
  }
}

void dfs(const Graph *g, int start) {
  int visited[MAX_VERTICES];                   // vector de nodos visitados
  memset(visited, 0, sizeof(visited));         // resetea vector a cero

  dfs_recursive(g, start, visited, 0);         // ejecuta DFS desde el nodo inicial

  for (int v = 0; v < g->V; v++)
    if (!visited[v])                           // revisa componentes desconectados
      dfs_recursive(g, v, visited, 0);         // explora nuevo componente
}
```

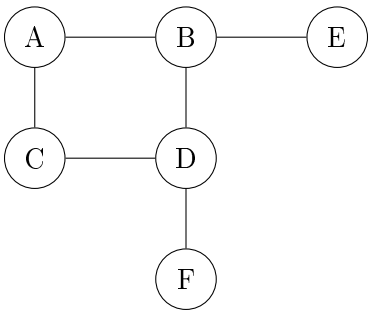
Listing 4: Recursive DFS with depth-indented output

11.1 Compile and Run

```
gcc -O0 -Wall -o dfs dfs.c
./dfs
```

12 Example Executions

12.1 Demo 1 — Undirected Connected Graph




```
DFS traversal starting from vertex [A]:

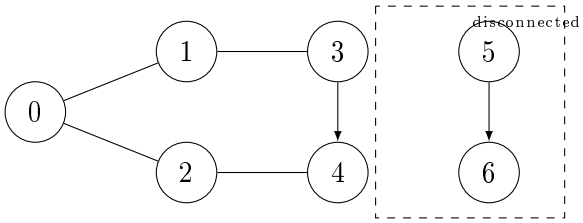
|-- [A]
  |-- [C]
```

```
    |-- [D]
      |-- [F]
      |-- [B]
        |-- [E]

DFS visit order: [A] -> [C] -> [D] -> [F] -> [B] -> [E]
```

The indentation reveals the **DFS tree**: each level of indentation corresponds to one recursion depth. Note that B is reached through D (not directly from A) because the adjacency list stores C before B for vertex A (edges were inserted with prepend).

12.2 Demo 2 — Directed Graph with Disconnected Component




```
DFS traversal starting from vertex [0]:

|-- [0]
  |-- [2]
    |-- [4]
  |-- [1]
    |-- [3]

[disconnected component, starting from 5]
|-- [5]
  |-- [6]

DFS visit order: [0] -> [2] -> [4] -> [1] -> [3] -> [5] -> [6]
```

The outer loop in `DFS-All` detects vertex 5, which was unreachable from 0, and starts a new DFS tree from it.

12.3 Demo 3 — Larger Graph (9 nodes)

```
DFS traversal starting from vertex [Alice]:

|-- [Alice]
  |-- [Carol]
    |-- [Frank]
      |-- [Iris]
        |-- [Hank]
          |-- [Eve]
            |-- [Bob]
```

```
            |-- [Dave]
              |-- [Grace]

DFS visit order:
  [Alice] -> [Carol] -> [Frank] -> [Iris] -> [Hank]
          -> [Eve] -> [Bob] -> [Dave] -> [Grace]
```

This demonstrates how DFS dives to the *deepest reachable* node before backtracking, producing a long single-branch traversal tree even in a graph with many cross connections.

13 Program Output — Execution Screenshots

The following figures show the actual terminal output produced by running `./dfs` on the three demo graphs.

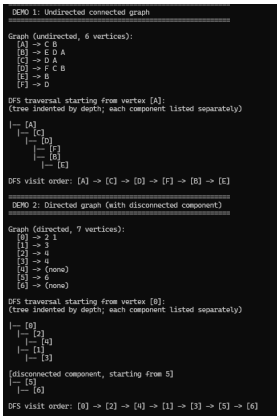


Figure: DFS output — Demo 1 (undirected) and Demo 2 (directed with disconnected component).

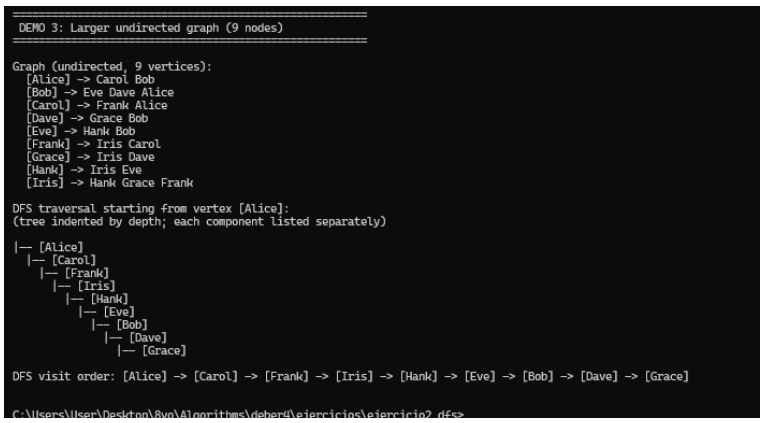


Figure: DFS output — Demo 3, larger undirected graph (9 nodes).

Part III: Breadth-First Search on a Graph

14 Problem Statement

Implement a **Breadth-First Search (BFS)** algorithm that traverses a graph and prints the value (label) of each node in the order it is visited. We must also justify the choice of graph data structure and explain what auxiliary data structure BFS requires.

15 Data Structure: Adjacency List + Queue

15.1 Graph Representation

We use the same **adjacency list** as in the DFS implementation. The justification is identical:

Property	Adjacency Matrix	**Adjacency List**
Space	$O(V^2)$	$O(V+E)$
BFS time	$O(V^2)$	$O(V+E)$
Neighbour iteration	$O(V)$	$O(\deg(u))$

15.2 The Queue — Why BFS Needs It

Unlike DFS (which uses the *call stack* as an implicit LIFO stack), BFS requires an explicit **FIFO queue**:

- BFS must process nodes in the order they were *discovered* (first-in, first-out) to guarantee that nodes at distance d are all visited before any node at distance $d+1$.
- A stack (LIFO) would instead produce DFS order — deeper nodes would be visited before the remaining shallower siblings.

We implement a **circular array queue** of capacity V. Each vertex is enqueued at most once, so the queue never overflows:

```
typedef struct {
  int data[MAX_VERTICES];
  int front, back, size;
} Queue;

static void queue_push(Queue *q, int v) {
  q->data[q->back] = v;
  q->back = (q->back + 1) % MAX_VERTICES;
  q->size++;
}

static int queue_pop(Queue *q) {
  int v = q->data[q->front];
```

```
14    q->front = (q->front + 1) % MAX_VERTICES;
15    q->size--;
16    return v;
17  }
```

Listing 5: Circular array queue

Both `push` and `pop` are $O(1)$.

16 BFS Algorithm

16.1 Idea

BFS explores the graph *level by level*: it visits all nodes at distance 1 from the source before any node at distance 2, and so on. This property makes BFS the basis for computing **shortest paths** in unweighted graphs.

16.2 Pseudocode

Algorithm 3 Breadth-First Search (queue-based)

Require: Graph G, source vertex s
Ensure: Visited array, distance array $dist[]$

```
1:  procedure BFS(G, s)
2:      initialise visited[ ] ← false,    dist[] ← ∞
3:      initialise Queue Q
4:      visited[s] ← true,    dist[s] ← 0
5:      ENQUEUE(Q, s)
6:      while Q is not empty do
7:          u ← DEQUEUE(Q)
8:          print label[u]
9:          for each neighbour v of u in G do
10:             if not visited[v] then
11:                 visited[v] ← true
12:                 dist[v] ← dist[u] + 1
13:                 ENQUEUE(Q, v)
14:             end if
15:         end for
16:     end while
17: end procedure
```

16.3 Complexity

- **Time**: $O(V + E)$ — every vertex enters the queue once ($O(V)$); every edge is examined once (directed) or twice (undirected) when scanning adjacency lists ($O(E)$).
- **Space**: $O(V)$ for the queue + $O(V)$ for visited/distance arrays.

17 C Implementation

```c
void bfs_component(const Graph *g, int start, int visited[], int dist[]) {
  Queue q;
  queue_init(&q);                          // inicializa la cola de exploracion

  visited[start] = 1;                      // marca nodo inicial como visitado
  dist[start] = 0;                         // distancia cero al nodo origen
  queue_push(&q, start);                   // encola el nodo de inicio

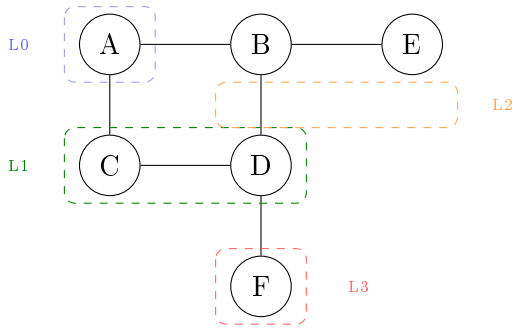
  while (!queue_is_empty(&q)) {            // mientras la cola no este vacia
    int u = queue_pop(&q);                 // extrae el siguiente nodo
    printf("  Level %d: [%s]\n", dist[u], g->label[u]); // imprime el nivel actual

    for (AdjNode *cur = g->heads[u]; cur; cur = cur->next) { // recorre vecinos
      int v = cur->dest;                   // obtiene vertice destino
      if (!visited[v]) {                   // si no ha sido visitado
        visited[v] = 1;                    // marca como visitado
        dist[v] = dist[u] + 1;             // asigna la distancia en saltos
        queue_push(&q, v);                 // agrega vecino a la cola
      }
    }
  }
}
```

Listing 6: BFS implementation with distance tracking

18 Example Executions

18.1 Demo 1 — Undirected Graph




```
BFS traversal starting from vertex [A]:
(level = shortest-path distance from source)

  Level 0: [A]
  Level 1: [C]
  Level 1: [B]
  Level 2: [D]
  Level 2: [E]
  Level 3: [F]

BFS visit order: [A] -> [C] -> [B] -> [D] -> [E] -> [F]

Shortest distances from [A]:
  [A]: 0 hop(s)    [B]: 1 hop(s)    [C]: 1 hop(s)
  [D]: 2 hop(s)    [E]: 2 hop(s)    [F]: 3 hop(s)
```

BFS correctly identifies that B and C are both at distance 1 from A, and that F is at distance 3 (the only path is $A \to B \to D \to F$ or $A \to C \to D \to F$).

18.2 Demo 2 — Directed Graph with Disconnected Component

```
BFS traversal starting from vertex [0]:

  Level 0: [0]
  Level 1: [2]
  Level 1: [1]
  Level 2: [4]
  Level 2: [3]

[disconnected component, starting from 5]
  Level 0: [5]
  Level 1: [6]

BFS visit order: [0] -> [2] -> [1] -> [4] -> [3] -> [5] -> [6]

Shortest distances from [0]:
```

```
  [0]: 0   [1]: 1   [2]: 1   [3]: 2   [4]: 2
  [5]: 0 (own component)   [6]: 1 (own component)
```

18.3 Demo 3 — Larger Graph (9 nodes)

```
BFS traversal starting from vertex [Alice]:

  Level 0: [Alice]
  Level 1: [Carol]
  Level 1: [Bob]
  Level 2: [Frank]
  Level 2: [Eve]
  Level 2: [Dave]
  Level 3: [Iris]
  Level 3: [Hank]
  Level 3: [Grace]

BFS visit order:
  [Alice] -> [Carol] -> [Bob] -> [Frank] -> [Eve]
          -> [Dave] -> [Iris] -> [Hank] -> [Grace]

Shortest distances from [Alice]:
  [Alice]:0  [Bob]:1  [Carol]:1  [Dave]:2
  [Eve]:2    [Frank]:2  [Grace]:3  [Hank]:3  [Iris]:3
```

In contrast to DFS (which dove 9 levels deep through a single chain), BFS reaches every node at its *true minimum distance*: all nodes at level 1 before any at level 2, and so on.

19 Program Output — Execution Screenshots

The following figures show the actual terminal output produced by running `./bfs` on the three demo graphs.

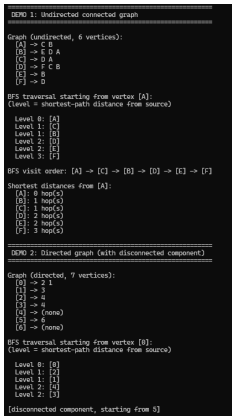


Figure: BFS output — Demo 1 (undirected) and Demo 2 (directed with disconnected component).

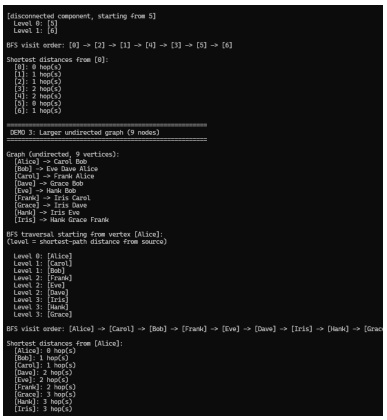


Figure: BFS output — Demo 3, larger undirected graph (9 nodes), shortest-path distances from Alice.

Conclusions and References

20 Discussion and Conclusions

20.1 Part I: 0/1 Knapsack Exhaustive Search

1. **Infeasibility of Brute Force for Large n**: The experimental results confirm the theoretical prediction: the running time grows *exponentially* with n ($\Theta(n \cdot 2^n)$). Incrementing n by 1 doubles the number of subsets evaluated.

   - For $n \leq 20$, the search completes in under 0.1 seconds.
   - At $n = 25$, time reaches ≈ 2.86 seconds.
   - At $n = 28$, time is 24.64 seconds (the largest problem solvable under 30s).
   - At $n = 29$, time jumps to 50.54 seconds, exceeding the 30-second budget.
   - For $n = 50$ (the full dataset instance), exhaustive search would require $2^{50} \approx 1.12 \times 10^{15}$ subsets, which would take over 30 years to execute.

   Therefore, exact polynomial algorithms (Dynamic Programming, Branch-and-Bound) or approximate heuristics (FPTAS, Greedy) are strictly necessary for practical instances.

2. **Maximum Solvable Size Within 30 Seconds**:

   $$\boxed{n^* = 28} \quad (2^{28} = 268{,}435{,}456 \text{ subsets evaluated})$$

   Even with a processor 100× faster, the exponential scale only grants an increase of $\lfloor \log_2 100 \rfloor = 6$ additional items ($n^* \approx 34$).

3. **Dataset and Implementation Justification**: The `kplib` dataset (Weakly Correlated family) provides academic credibility and reproducible benchmarks. C was chosen due to minimal memory overhead, zero runtime garbage collection, direct bitwise manipulation via `uint64_t`, and high-resolution timing with `clock_gettime`.

20.2 Part II & III: Graph Search Algorithms (DFS vs. BFS)

1. **Data Structure Efficiency**: Both DFS and BFS operate optimally in $O(V + E)$ time when implemented with an **Adjacency List**, touching only actual edges. In contrast, an **Adjacency Matrix** forces $O(V^2)$ iterations over non-existent edges, which is inefficient for sparse graphs ($E \ll V^2$).

2. **Auxiliary Data Structures**:

   - **DFS** relies on a **LIFO Stack** (either explicit or implicit via function recursion). It explores deeply down a single path before backtracking.
   - **BFS** strictly requires a **FIFO Queue**. By processing vertices in order of discovery, it explores level-by-level.

3. **Comparative Analysis**:

Property	**BFS**	**DFS**
Auxiliary structure	Queue (FIFO)	Stack / call stack (LIFO)
Visit order	Level-by-level	Deepest branch first
Shortest path	**Yes** (unweighted)	No
Cycle detection	Yes	Yes
Time complexity	$O(V+E)$	$O(V+E)$
Space complexity	$O(V)$ queue	$O(V)$ stack depth
Primary use cases	Shortest paths, minimum hops	Topological sort, connected components
Demo 3 — same graph (9 nodes), different visit order:		
DFS order	Alice → Carol → Frank → Iris → Hank → Eve → Bob → Dave → Grace	
BFS order	Alice → Carol → Bob → Frank → Eve → Dave → Iris → Hank → Grace	

20.3 Summary

This assignment evaluated both combinatorial exhaustive search and core graph search algorithms. While exhaustive search illustrates the fundamental limits of brute-force exponential algorithms ($\Theta(2^n)$), DFS and BFS demonstrate how structural graph properties and appropriate data structures (Adjacency List + Queue/Stack) achieve linear complexity ($O(V+E)$) for structural graph problems.

References

1. Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack Problems*. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-24777-7

2. Pisinger, D. (1995). *Algorithms for Knapsack Problems*. PhD thesis, University of Copenhagen.

3. likr. (2014). *kplib: Test instances for knapsack problems*. GitHub. https://github.com/likr/kplib

4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (BFS: Section 20.2; DFS: Section 20.3; Graph representations: Section 20.1)